

PROJECT REPORT: GUI BASED CAR RACING GAME USING JAVA

Motive Of Implementation: 2D Game Development using Java Swing & AWT

Implementation Files: Week2.java through Week9.java

1. Project Overview & Objectives

The objective of this project was to design and construct a fully functional 2D arcade-style Car Racing Game using core Java components. Developed iteratively across multiple phases (Weeks 2 through 9), the application integrates graphical rendering, state machine handling, real-time user input processing, collision detection physics, score/timer tracking, and embedded audio controls.

2. Overall System Architecture & Evolution

Development Phase	Key Implementation Focus	Core Java Features Used To Implemen
Weeks 2–4	Window Creation & Base Track GUI	JFrame, JPanel, Graphics, Color
Weeks 5–6	Object Rendering & Movement	Graphics2D, fillRoundRect(), drawOval(), Basic Loops
Week 7	Game Loop & Animation	javax.swing. Timer, ActionListener, Object Offset Logic
Week 8	Physics, Controls & Audio Integration	KeyListener, java.awt.Rectangle.intersects(), javax.sound.sampled.Clip
Week 9	States (Welcome/GameOver), UI & Restart	Boolean States (isWelcomeScreen, isGameOver), Font Metrics, Resource Reset

3. Core Game Mechanics & System Components

3.1 Graphical Interface & Rendering Engine

- **Screen Dimensions:** Fixed at 600px width × 700px height (WIDTH = 600, HEIGHT = 700).
- **Road & Track Aesthetics:** Renders a central dark gray driving track with white side boundaries and animated lane dividers.
- **Custom Vehicle Renderer (drawCar):** Uses vector shapes (fillRoundRect, fillOval, fillRect) to construct detailed player and enemy cars complete with body chassis, windshields, wheels, headlights, and tail lights.

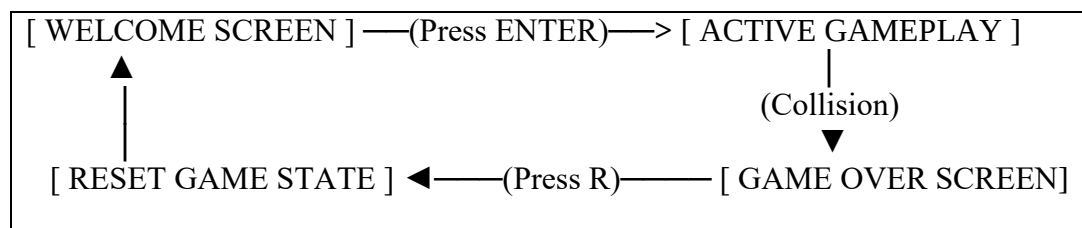
3.2 Game Loop & Real-Time Animation

- **FPS Controller:** Driven by a javax.swing.Timer executing at a 16ms interval (~60 FPS).
- **Scrolling Track Effect:** roadOffsetY increments continuously by 5px per frame to simulate forward vehicle velocity.
- **Enemy Spawning:** Enemy cars move downward (enemyY += 6). When they travel past the bottom screen (enemyY > 700), they reset to top position (enemyY = -120) at a randomized X-coordinate on the track (160 + random.nextInt(220)).

3.3 Physics & Collision Detection

- Uses AWT bounding box intersections for pixel-accurate collision.
- Rectangle playerRect = new Rectangle(playerX, playerY, 60, 110)
- Rectangle enemyRect = new Rectangle(enemyX, enemyY, 60, 110)
- When playerRect.intersects(enemyRect) returns true, game execution immediately halts, background engine audio stops, and crash audio triggers.

3.4 Multi-State Machine Flow



1. **Welcome Screen (isWelcomeScreen = true):** Displays centered title in cyan, control instructions, preview car, and loops welcome.wav.
2. **Active Gameplay (isWelcomeScreen = false, isGameOver = false):** Processes player input, moves road and random enemy vehicles, updates survival timer in second and score (+10 per dodge), and loops run.wav.
3. **Game Over Screen (isGameOver = true):** Pauses timer, stops engine sound, plays crash.wav, and displays final score gained, shows total survived time, and implements restart prompt.

3.5 Audio Asset System

Managed through class path resource paths (/weeks/filename.wav) to prevent file loading issues across operating systems:

- **welcome.wav:** Continuous loop on welcome screen.
- **run.wav:** Continuous loop during active driving.
- **score.wav:** One-shot clip played when successfully dodging an enemy car.
- **crash.wav:** One-shot clip played on vehicle impact.

4. Hardware & Software Requirements

- **Operating System:** Cross-platform (Like: Windows)
- **Development Environment (IDE):** NetBeans
- **Java SDK:** JDK 8 or higher.
- **Audio Requirements:** Standard audio output device (external or implemented internal)

5. Controls & User Guide

- **ENTER Key:** Start game from the Welcome Screen.
- **LEFT Arrow Key:** Move player car left (bounded to road margin at playerX > 150).
- **RIGHT Arrow Key:** Move player car right (bounded to road margin at playerX < 390).
- **R Key:** Restart game after Game Over.

6. Discussion

The development of the 2D Car Racing Game demonstrates the practical application of object-oriented programming (OOP) and graphical user interface (GUI) design principles in Java programming language. Over the course of nine weeks, the application evolved from basic window rendering to an interactive, state-driven arcade game successful game implementation.

Key Technical Achievements

- **State Machine Architecture:** Implementing distinct boolean flags (isWelcomeScreen and isGameOver) made it straightforward to isolate screen rendering, sound triggers, and perfect input processing. This design prevents from control inputs or audio clips from firing at incorrect times and ensure perfect time implementation.
- **Resource Management & Sound:** Transitioning to classpath resource paths (/weeks/filename.wav) resolved file path resolution issues across different IDE environments and operating systems. Furthermore, using separate Clip objects allowed smooth background looping alongside one-shot sound effects without causing audio latency.
- **Real-time Collision Logic:** Utilizing AWT's Rectangle.intersects() provided a lightweight, reliable bounding-box method for real-time collision detection at 60 FPS. By continuously comparing the dynamic boundary dimensions of the player's vehicle against incoming enemy cars during each timer tick, the system evaluates overlaps instantaneously without impacting game performance. This ensures immediate state transitions to the Game Over screen perfectly.

Limitations & Potential Enhancements

While the project meets all functional requirements, future iterations could expand the gameplay experience:

1. **Dynamic Difficulty Through Creating Levels:** Increasing vehicle speed or narrowing the road as time progresses by implementing various level options to make the game more challenging over time.
2. **Multiple Obstacles:** Spawning multiple enemy cars simultaneously across different lanes.
3. **High Score Persistence:** Saving top scores locally to a file or database so player achievements persist after closing the application.

7. Conclusion

The Java Car Racing Game project successfully demonstrates how foundational Java concepts—such as event handling, multi-threading via Swing timers, vector graphics, and audio processing—can be combined to build an engaging 2D game. By incrementally building the project from track rendering (Week 2) to sound integration (Week 8) and state-based game management (Week 9), the software architecture remains clean, maintainable, and robust. The resulting application provides an attractive welcome interface, smooth controls, reactive sound effects, and a seamless restart mechanism, meeting all target objectives.